Índices y planes de ejecución

Valentina Gonzalez Diaz

Valentina Rodriguez Castro

Raquel Lopez Aristizabal

Código:

1094898396

1016833647

1091884575



Profesor: Carolina Londoño Idarraga

Universidad del Quindío

Facultad de Ingeniería

Programa Ingeniería de Sistemas y Computación

Bases de Datos II

Armenia, 2026

Se identificaron tres consultas frecuentes del sistema TurismoUQ que se resolvían leyendo tablas completas, y una cuarta donde un índice no mejora el plan. Para cada una se capturó el plan de ejecución antes y después de crear el índice, y se comparó el *Cost* estimado por el optimizador de Oracle.

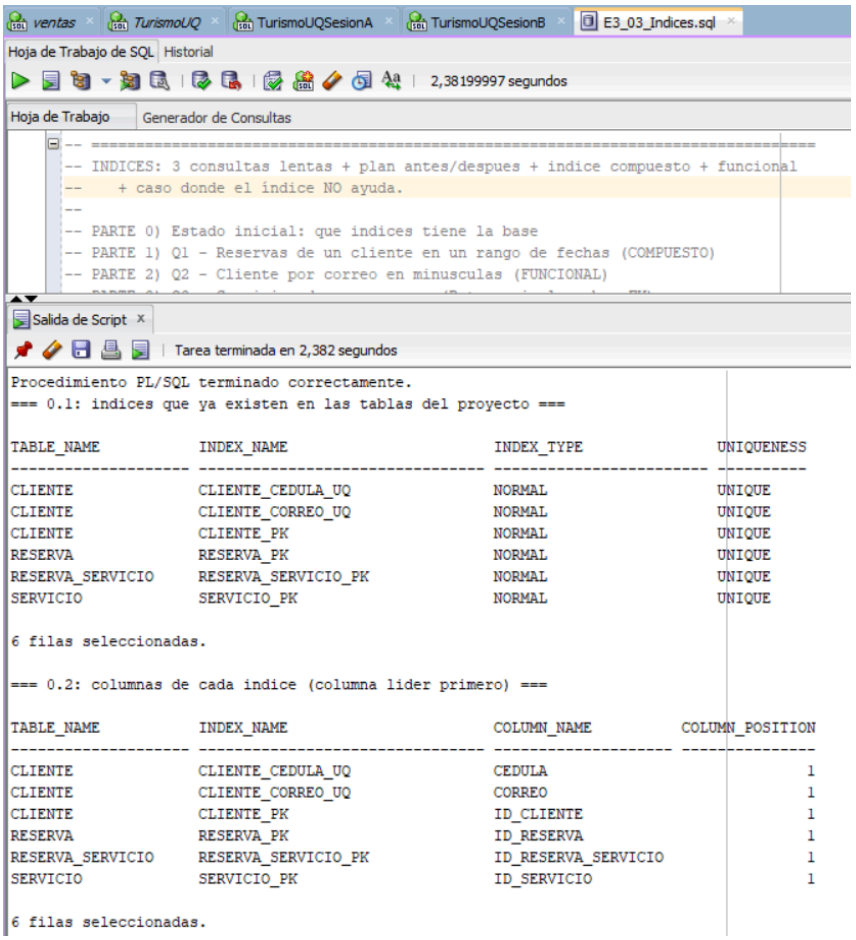
Los planes se obtuvieron con EXPLAIN PLAN FOR y DBMS_XPLAN.DISPLAY con el formato 'BASIC +COST +ROWS +PREDICATE'. El *Cost* es una unidad relativa que estima el optimizador, no un tiempo en segundos.

1. **Estado inicial de los índices**

Antes de crear los indices, las tablas solo tenían los que Oracle crea automáticamente por las restricciones PK y UNIQUE.

- reserva.id_cliente y reserva_servicio.id_reserva son FK y no tienen índice.

- cliente.correo si tiene un índice pero guarda el correo tal cual está escrito, no en minúsculas.



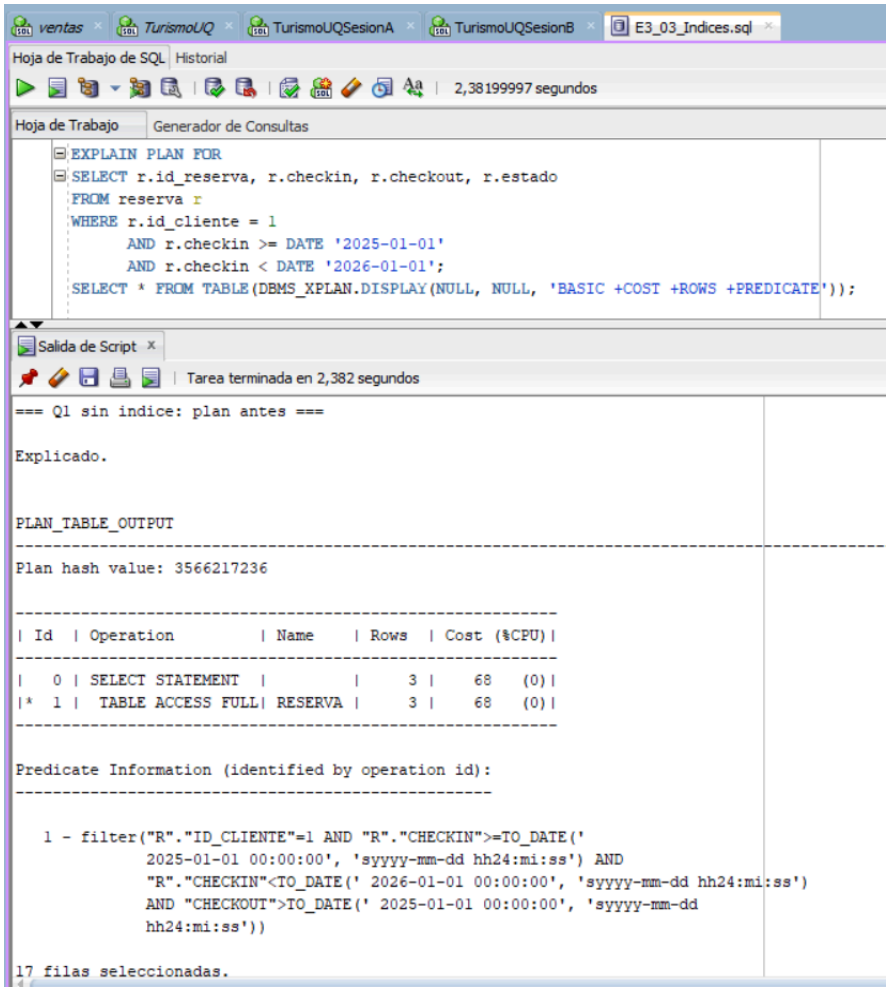
2. **Las consultas**

Q1: Reservas de un cliente en un rango de fechas (índice compuesto)

Consulta plan antes:

```
PROMPT === Q1 sin indice: plan antes ===
EXPLAIN PLAN FOR
SELECT r.id_reserva, r.checkin, r.checkout, r.estado
FROM reserva r
WHERE r.id_cliente = 1
      AND r.checkin >= DATE '2025-01-01'
      AND r.checkin < DATE '2026-01-01';
SELECT  *  FROM  TABLE(DBMS_XPLAN.DISPLAY(NULL,  NULL,  'BASIC
+COST +ROWS +PREDICATE'));
```

Necesidad: recepción consulta las reservas de un cliente durante un año.



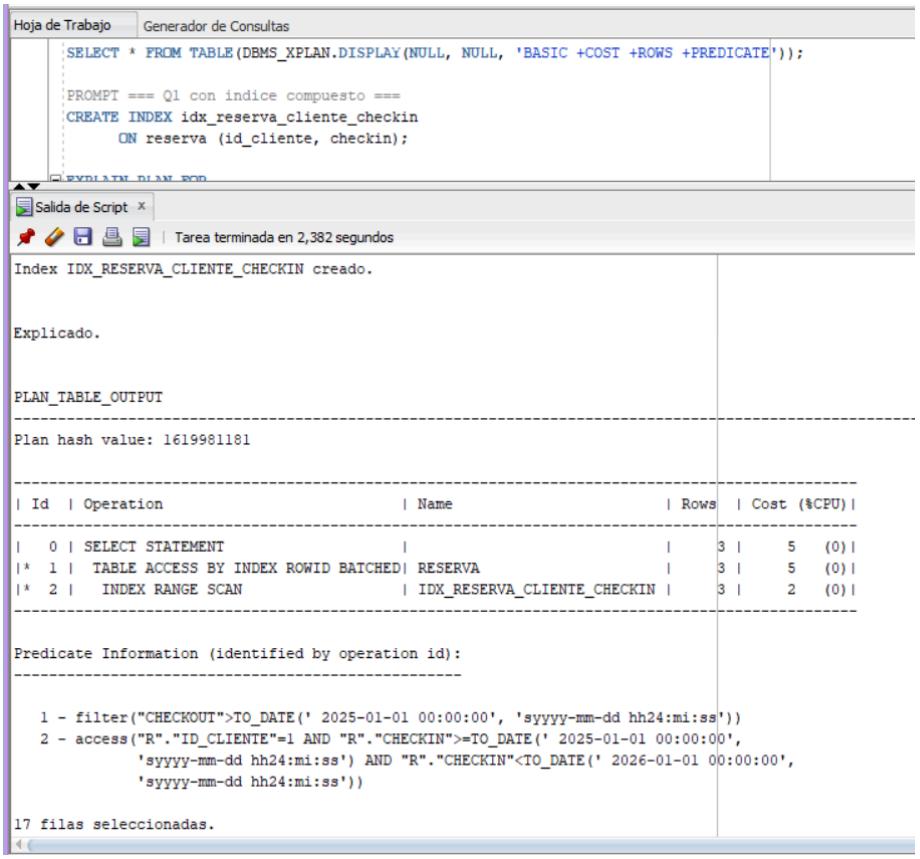
Índice creado:

```
PROMPT === Q1 con indice compuesto ===
CREATE INDEX idx_reserva_cliente_checkin
      ON reserva (id_cliente, checkin);
```

Consulta plan después:

```
EXPLAIN PLAN FOR
SELECT r.id_reserva, r.checkin, r.checkout, r.estado
FROM reserva r
WHERE r.id_cliente = 1
      AND r.checkin >= DATE '2025-01-01'
      AND r.checkin < DATE '2026-01-01';
SELECT  *  FROM  TABLE(DBMS_XPLAN.DISPLAY(NULL,  NULL,  'BASIC
+COST +ROWS +PREDICATE'));

PROMPT  ===  Q1  control:  la  consulta  devuelve  las  mismas  filas
===
SELECT COUNT(*) AS filas_devueltas
FROM reserva r
WHERE r.id_cliente = 1
      AND r.checkin >= DATE '2025-01-01'
      AND r.checkin < DATE '2026-01-01';
```



La columna líder es id_cliente, que se filtra por igualdad, y la segunda es checkin, que se filtra por rango. Con la igualdad primero, Oracle puede acotar y recorrer el rango. Un índice solo sobre id_cliente sirve, pero leería todas las reservas del cliente y filtraría las fechas después.

Q2: Cliente por correo sin importar mayúsculas (índice funcional)

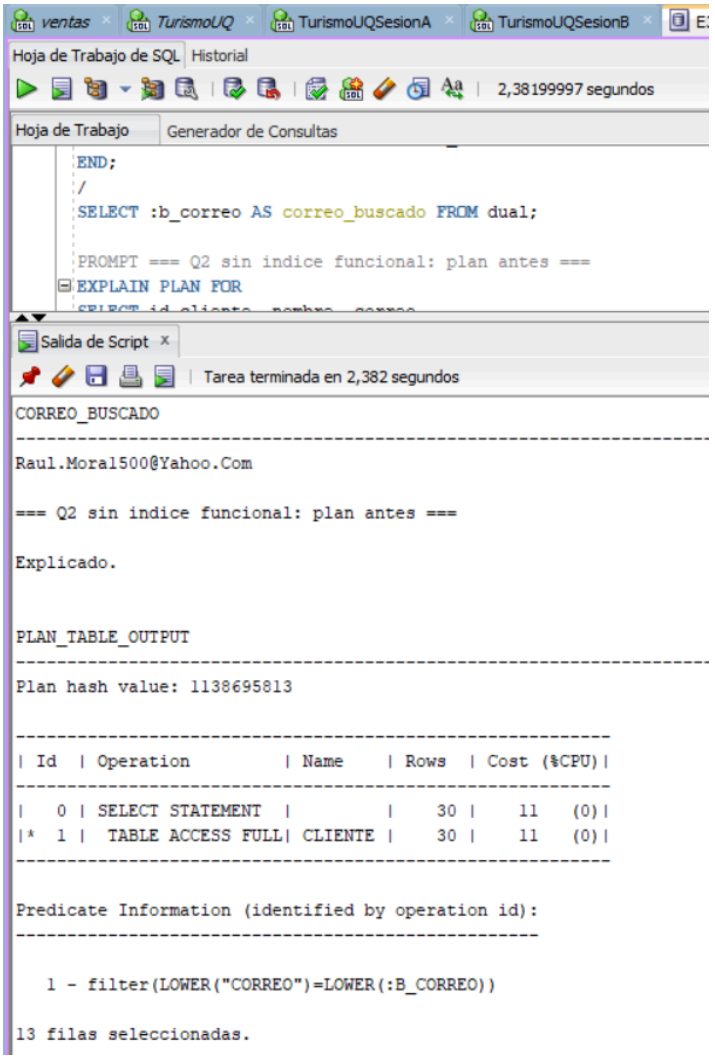
Consulta plan antes:

```
VARIABLE b_correo VARCHAR2(80)

-- Correo del cliente 1500 con mayusculas mezcladas
BEGIN
       SELECT INITCAP(correo) INTO :b_correo FROM cliente WHERE
id_cliente = 1500;
END;
/
SELECT :b_correo AS correo_buscado FROM dual;

PROMPT === Q2 sin indice funcional: plan antes ===
EXPLAIN PLAN FOR
SELECT id_cliente, nombre, correo
FROM cliente
WHERE LOWER(correo) = LOWER(:b_correo);
SELECT   *   FROM   TABLE(DBMS_XPLAN.DISPLAY(NULL,   NULL,   'BASIC
+COST +ROWS +PREDICATE'));
```

Necesidad: En el registro o el inicio de sesión, el usuario puede escribir su correo con cualquier combinación de mayúsculas y minúsculas.



Índice creado:

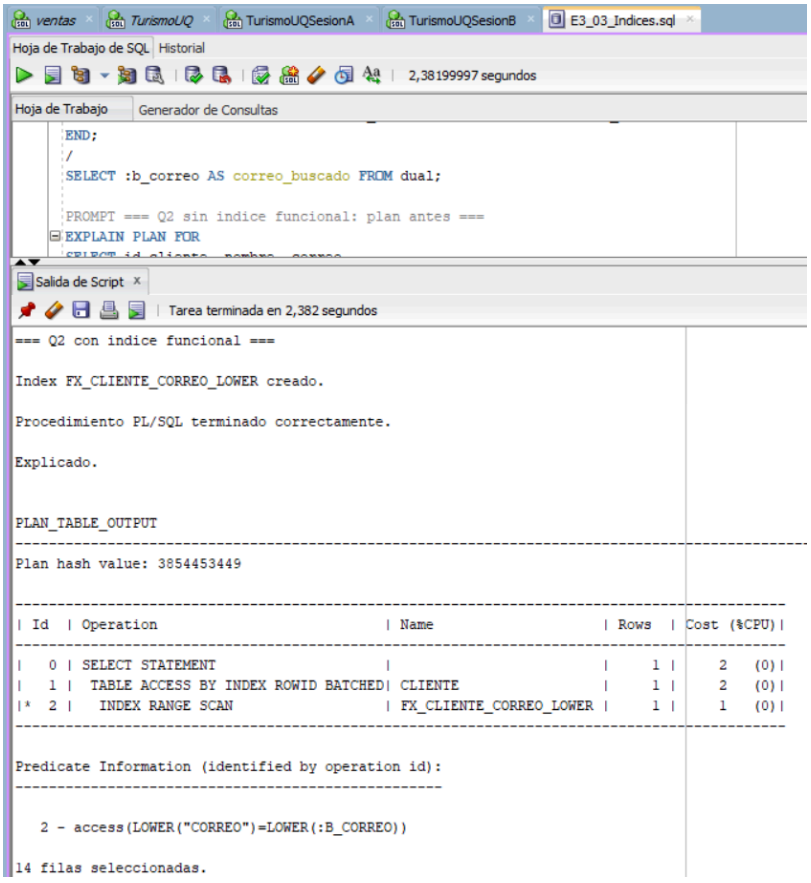
```
PROMPT === Q2 con indice funcional ===
CREATE INDEX fx_cliente_correo_lower
      ON cliente (LOWER(correo));

BEGIN
      DBMS_STATS.GATHER_TABLE_STATS(USER, 'CLIENTE', cascade
=> TRUE);
END;
/
```

Consulta plan después:

```
EXPLAIN PLAN FOR
SELECT id_cliente, nombre, correo
FROM cliente
WHERE LOWER(correo) = LOWER(:b_correo);
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(NULL, NULL, 'BASIC
+COST +ROWS +PREDICATE'));

PROMPT === Q2 control: mismo resultado, con y sin indice ===
SELECT COUNT(*) AS filas_devueltas
FROM cliente
WHERE LOWER(correo) = LOWER(:b_correo);
```



Con el índice funcional el plan pasa a INDEX RANGE SCAN sobre FX_CLIENTE_CORREO_LOWER y el Cost baja de 11 a 2. El índice guarda el valor ya en minúsculas, por eso la comparación se resuelve directamente.

Q3: Servicios de una reserva (B-tree simple sobre llave foránea)

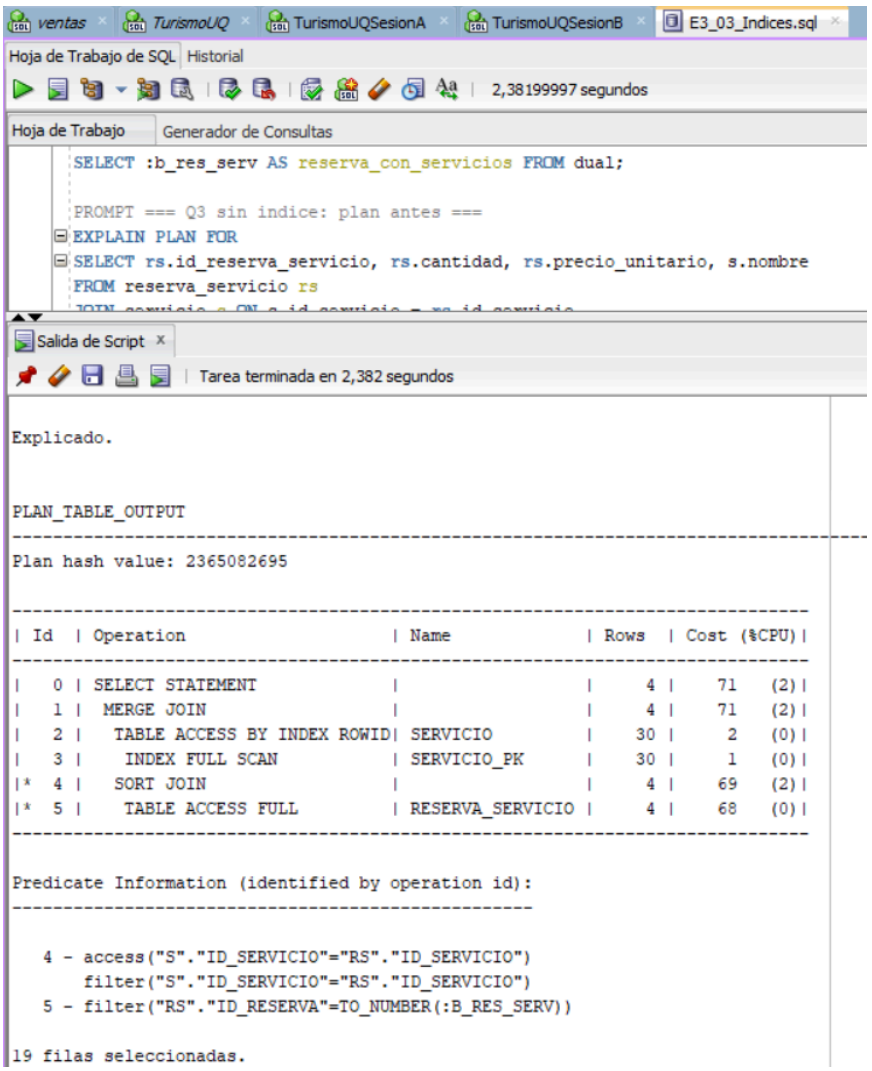
Consulta plan antes:

```
VARIABLE b_res_serv NUMBER

BEGIN
      SELECT id_reserva INTO :b_res_serv
      FROM (SELECT id_reserva
            FROM reserva_servicio
            GROUP BY id_reserva
            ORDER BY COUNT(*) DESC, id_reserva)
      WHERE ROWNUM =1;
END;
/
SELECT :b_res_serv AS reserva_con_servicios FROM dual;

PROMPT === Q3 sin indice: plan antes ===
EXPLAIN PLAN FOR
SELECT          rs.id_reserva_servicio,          rs.cantidad,
rs.precio_unitario, s.nombre
FROM reserva_servicio rs
JOIN servicio s ON s.id_servicio = rs.id_servicio
WHERE rs.id_reserva = :b_res_serv;
SELECT  *  FROM  TABLE(DBMS_XPLAN.DISPLAY(NULL,  NULL,  'BASIC
+COST +ROWS +PREDICATE'));
```

Necesidad: Mostrar o liquidar los servicios contratados en una reserva.



Índice creado:

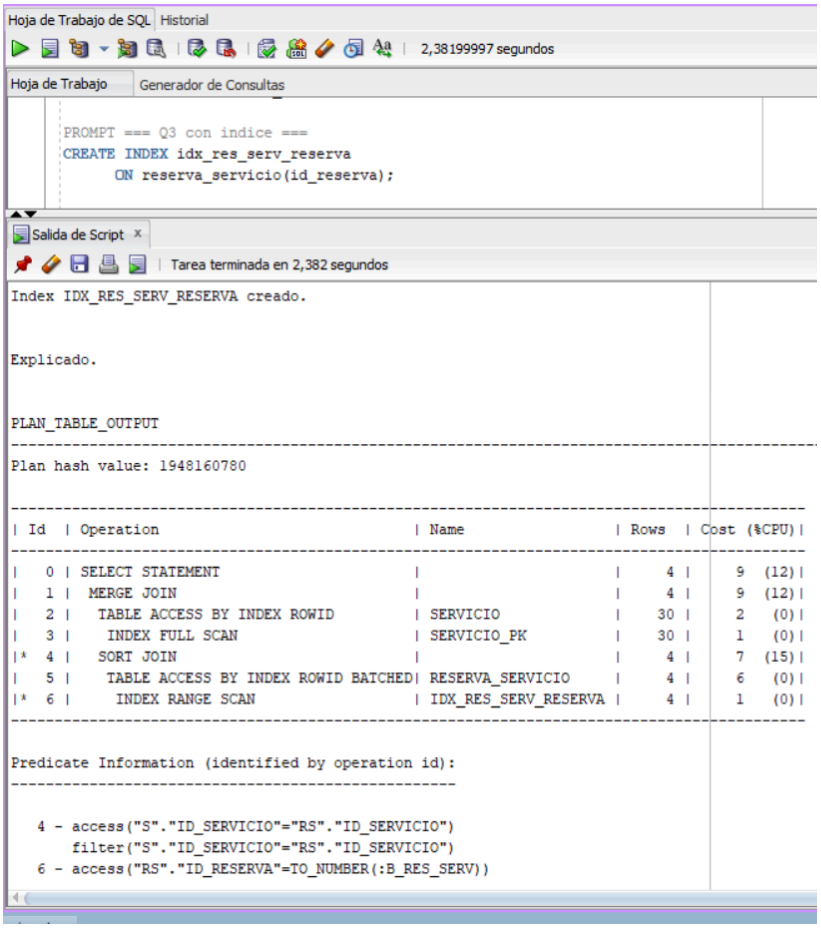
```
PROMPT === Q3 con indice ===
CREATE INDEX idx_res_serv_reserva
      ON reserva_servicio(id_reserva);
```

Consulta plan después:

```
EXPLAIN PLAN FOR
SELECT rs.id_reserva_servicio, rs.cantidad,
rs.precio_unitario, s.nombre
FROM reserva_servicio rs
JOIN servicio s ON s.id_servicio = rs.id_servicio
WHERE rs.id_reserva = :b_res_serv;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY(NULL, NULL, 'BASIC
+COST +ROWS +PREDICATE'));

PROMPT === Q3 control: filas devueltas (mismas con y sin
indice) ===
```

```
SELECT rs.id_reserva_servicio, rs.cantidad,
rs.precio_unitario, s.nombre
FROM reserva_servicio rs
JOIN servicio s ON s.id_servicio = rs.id_servicio
WHERE rs.id_reserva = :b_res_serv;
```

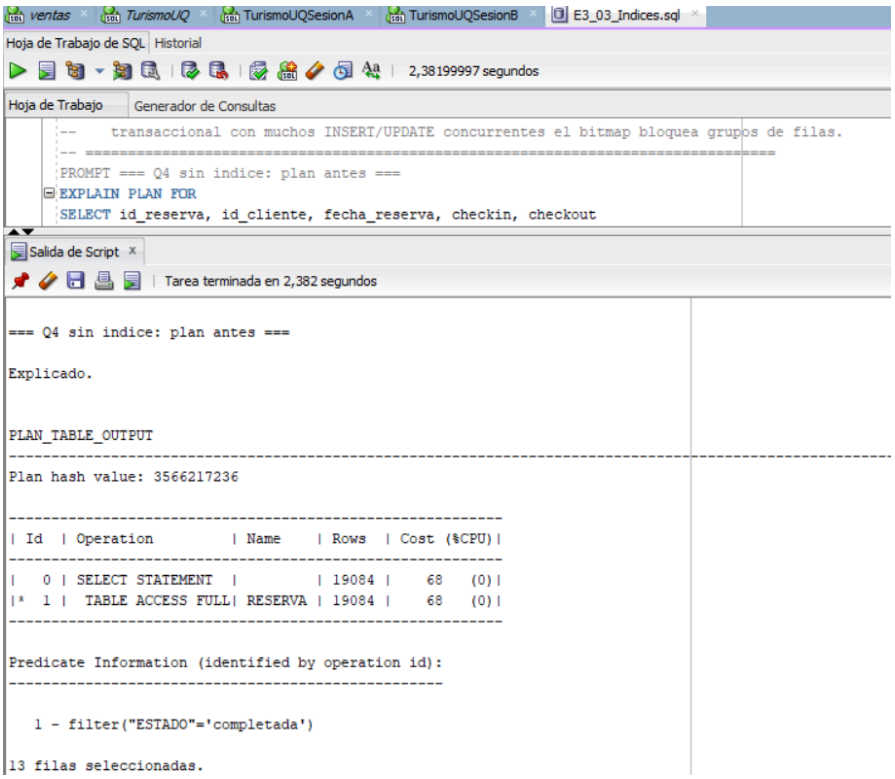


Con el índice, la lectura completa se reemplaza por un INDEX RANGE SCAN y el Cost total baja de 71 a 9.

Q4: Reservas por estado (el índice no ayuda)

Consulta plan antes:

```
PROMPT === Q4 sin indice: plan antes ===
EXPLAIN PLAN FOR
SELECT id_reserva, id_cliente, fecha_reserva, checkin,
checkout
FROM reserva
WHERE estado = 'completada';
SELECT * FROM TABLE (DBMS_XPLAN.DISPLAY(NULL, NULL, 'BASIC
+COST +ROWS +PREDICATE'));
```



Índice creado:

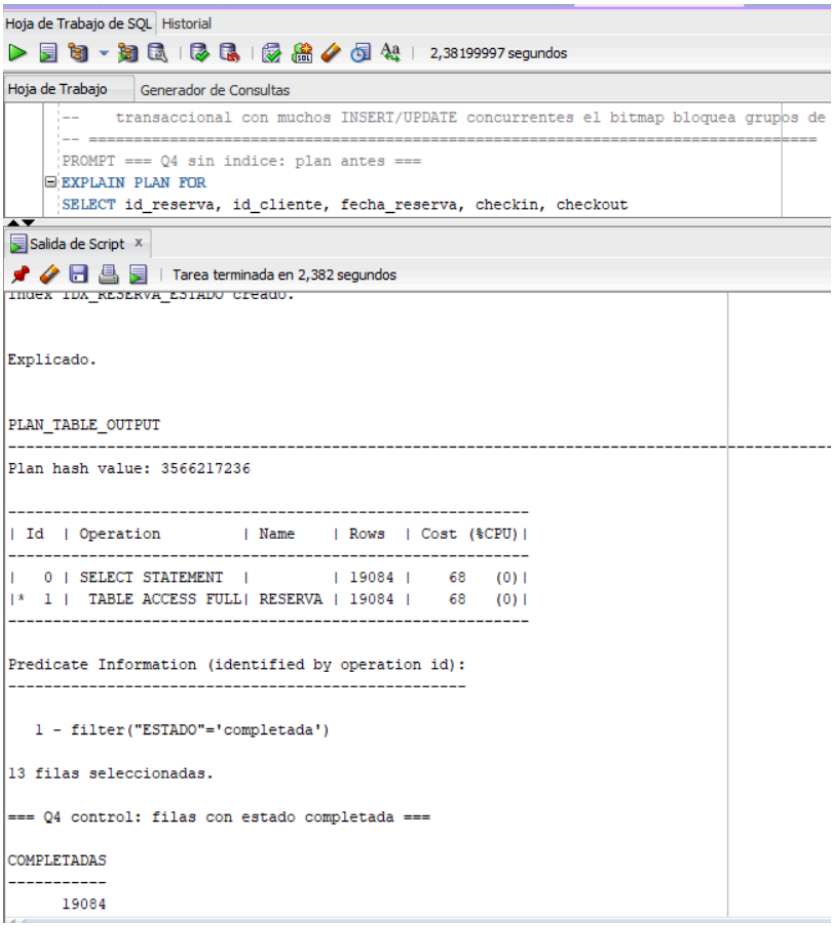
```
PROMPT === Q4 con indice B-tree sobre estado: plan despues ===
CREATE INDEX idx_reserva_estado ON reserva (estado);
```

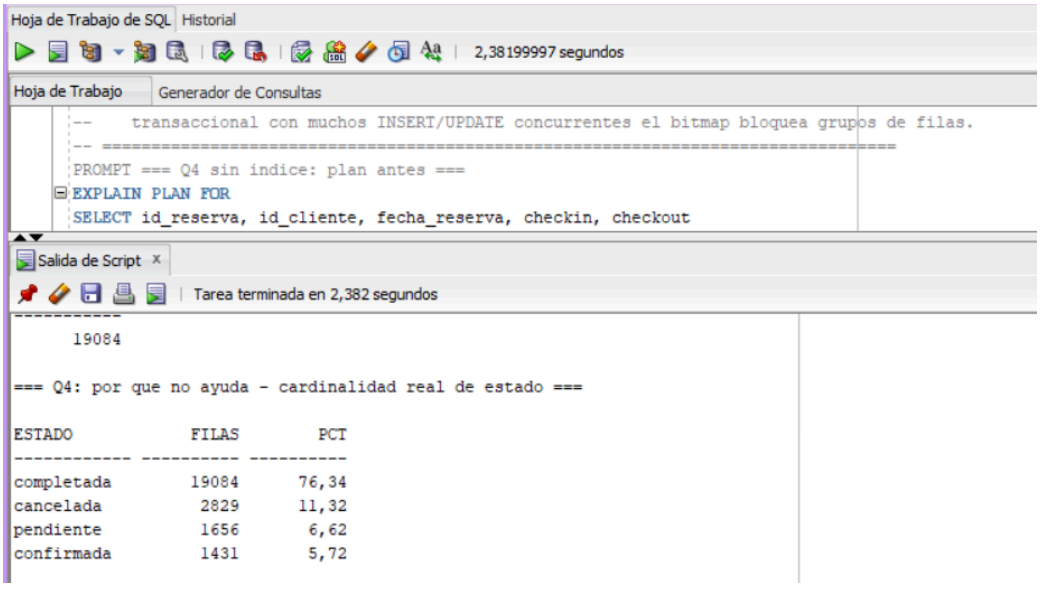
Consulta plan después:

```
EXPLAIN PLAN FOR
SELECT id_reserva, id_cliente, fecha_reserva, checkin,
checkout
FROM reserva
WHERE estado = 'completada';
SELECT * FROM TABLE (DBMS_XPLAN.DISPLAY(NULL, NULL, 'BASIC
+COST +ROWS +PREDICATE'));

PROMPT === Q4 control: filas con estado completada ===
SELECT COUNT(*) AS completadas FROM reserva WHERE estado =
'completada';

PROMPT === Q4: por que no ayuda - cardinalidad real de estado
===
SELECT estado, COUNT(*) AS filas,
       ROUND(100 * COUNT(*) / (SELECT COUNT(*) FROM reserva),
2) AS pct
FROM reserva
GROUP BY estado
ORDER BY filas DESC;
```





1. Estado tiene solo 4 valores distintos, es decir, baja selectividad y ‘completada’ es el 76% de la tabla.
2. Para usar el índice, Oracle tendría que saltar del índice a la tabla fila por fila para traer id_cliente, fecha_reserva, checkin y checkout al menos unas 19.000 veces
3. Eso cuesta más que leer la tabla completa una sola vez. Cuando una consulta devuelve gran parte de la tabla, la lectura completa es más barata.
4. Aunque no se este usando, el índice ocupa espacio y hace más lentos los INSERT, UPDATE y DELETE de reserva, una tabla que es transaccional.

5. Un índice bitmap es una opción típica si son pocos valores distintos, pero bloquea grupos de filas y no es conveniente para una tabla con muchas escrituras simultáneas.

Resumen de resultados:

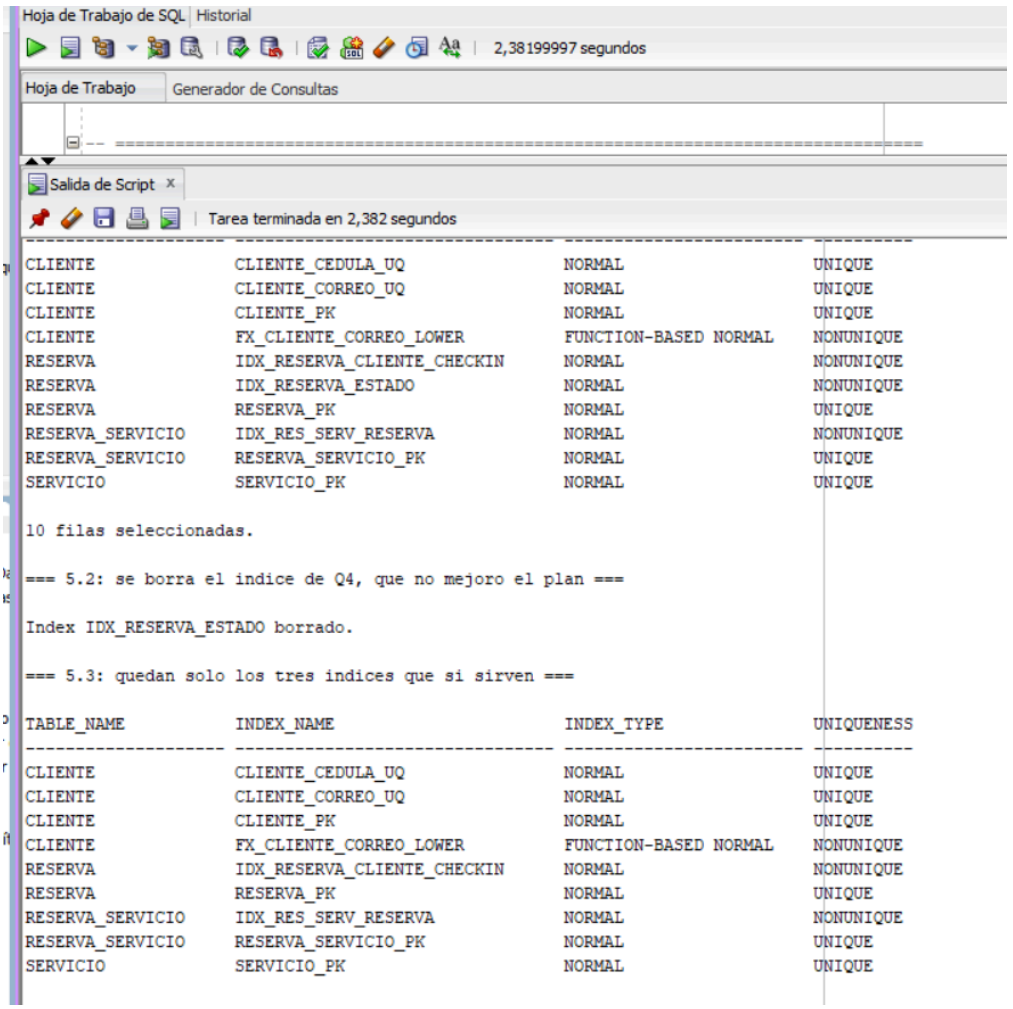
Consulta	Tipo índice	Cost antes	Cost después	Mejora
Q1	Compuesto (id_cliente, checkin)	68	5	≈ 93%
Q2	Funcional LOWER(correo)	11	2	≈ 82%
Q3	B-tree sobre FK id_reserva	71	9	≈ 87%
Q4	Btree sobre estado	68	68	0%

Índices finales:

```
PROMPT === 5.1: indices finales (los tres utiles +
idx_reserva_estado de Q4) ===
SELECT table_name, index_name, index_type, uniqueness
FROM user_indexes
WHERE table_name IN ('RESERVA','CLIENTE', 'RESERVA_SERVICIO',
'SERVICIO')
ORDER BY table_name, index_name;

PROMPT === 5.2: se borra el indice de Q4, que no mejoro el
plan ===
DROP INDEX idx_reserva_estado;

PROMPT === 5.3: quedan solo los tres indices que si sirven ===
SELECT table_name, index_name, index_type, uniqueness
FROM user_indexes
WHERE table_name IN ('RESERVA','CLIENTE', 'RESERVA_SERVICIO',
'SERVICIO')
ORDER BY table_name, index_name;
```



Hoja de Trabajo de SQL | Historial

2,38199997 segundos

Hoja de Trabajo | Generador de Consultas

```
-- ================================================================================
```

Salida de Script

Tarea terminada en 2,382 segundos

```
-------------------- ------------------------------ ------------------------ ----------
CLIENTE              CLIENTE_CEDULA_UQ              NORMAL                   UNIQUE
CLIENTE              CLIENTE_CORREO_UQ              NORMAL                   UNIQUE
CLIENTE              CLIENTE_PK                     NORMAL                   UNIQUE
CLIENTE              FX_CLIENTE_CORREO_LOWER        FUNCTION-BASED NORMAL    NONUNIQUE
RESERVA              IDX_RESERVA_CLIENTE_CHECKIN    NORMAL                   NONUNIQUE
RESERVA              IDX_RESERVA_ESTADO             NORMAL                   NONUNIQUE
RESERVA              RESERVA_PK                     NORMAL                   UNIQUE
RESERVA_SERVICIO     IDX_RES_SERV_RESERVA           NORMAL                   NONUNIQUE
RESERVA_SERVICIO     RESERVA_SERVICIO_PK            NORMAL                   UNIQUE
SERVICIO             SERVICIO_PK                    NORMAL                   UNIQUE

10 filas seleccionadas.

=== 5.2: se borra el indice de Q4, que no mejoro el plan ===

Index IDX_RESERVA_ESTADO borrado.

=== 5.3: quedan solo los tres indices que si sirven ===

TABLE_NAME           INDEX_NAME                     INDEX_TYPE               UNIQUENESS
-------------------- ------------------------------ ------------------------ ----------
CLIENTE              CLIENTE_CEDULA_UQ              NORMAL                   UNIQUE
CLIENTE              CLIENTE_CORREO_UQ              NORMAL                   UNIQUE
CLIENTE              CLIENTE_PK                     NORMAL                   UNIQUE
CLIENTE              FX_CLIENTE_CORREO_LOWER        FUNCTION-BASED NORMAL    NONUNIQUE
RESERVA              IDX_RESERVA_CLIENTE_CHECKIN    NORMAL                   NONUNIQUE
RESERVA              RESERVA_PK                     NORMAL                   UNIQUE
RESERVA_SERVICIO     IDX_RES_SERV_RESERVA           NORMAL                   NONUNIQUE
RESERVA_SERVICIO     RESERVA_SERVICIO_PK            NORMAL                   UNIQUE
SERVICIO             SERVICIO_PK                    NORMAL                   UNIQUE
```